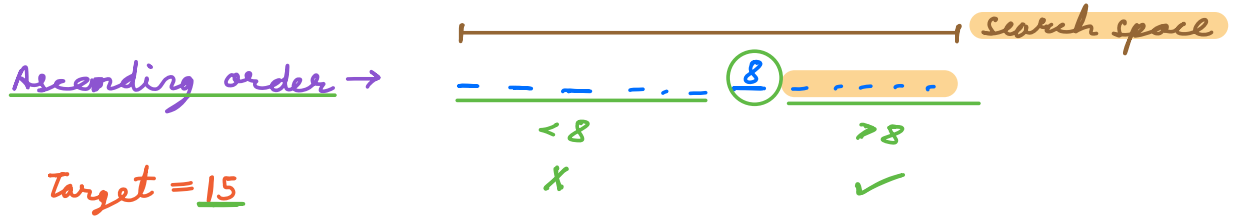


Search for a target value in a organized search space.



Q $\rightarrow$ Given a sorted array of integers.
Search for a target value & return its index, if not present return -1.

$A = [1, 3, 5, 10, 12]$ target = 10 Ans = 3
 0 1 2 3 4

Bruteforce $\rightarrow \forall i$ check if $A[i] == \text{target}$. TC = $O(N)$ SC = $O(1)$

search space $\rightarrow$ L — R
 [0 13]

Binary Search

target = 17

$A = [1, 3, 5, 7, 9, 10, 11, 13, 15, 17, 19, 30, 35, 40]$
 0 1 2 3 4 5 6 7 8 9 10 11 12 13
 L R

1) $L = 0$ $R = 13$ $\text{Mid} = \frac{0+13}{2} = 6$

$17 > A[\text{Mid}]$

$\Rightarrow$ L — R
 7 13 $L = 7$

2) $L = 7$ $R = 13$ $\text{Mid} = \frac{7+13}{2} = 10$

$17 < A[\text{Mid}] \Rightarrow$ L — R
 7 9 $R = 9$

3) $L = 7$ $R = 9$ $\text{Mid} = \frac{7+9}{2} = 8$ $17 > A[\text{Mid}] \Rightarrow$ L — R
 9 9 $L = 9$

4) $L = 9$ $R = 9$ $\text{Mid} = \frac{9+9}{2} = 9$ $17 = A[\text{Mid}] \Rightarrow \text{Ans} = \text{Mid} \checkmark$

```

L = 0    R = N-1    // Define Search Space ①
while (L <= R) {
    Mid = (L+R) / 2    // Mid = L + (R-L) / 2
    if (A[Mid] == Target) // check if mid is answer ②
        return Mid
    if (A[Mid] < Target)
        L = Mid + 1
    else
        R = Mid - 1
}

// Decide going left or right ③
TC = O(log2(N))
N →  $\frac{N}{2} \rightarrow \frac{N}{4} \dots \frac{N}{2^k} = 1 \Rightarrow N = 2^k \Rightarrow k = \log_2(N)$ 
SC = O(1)
return -1 ✓

```

Q → Given a sorted array of integers.

Search for a target value & return its first index, if not present return -1. (Duplicates in array)

$TC = O(N)$
 A = [2, 2, 5, 5, 5, 5, 8, 10, 10, 13, 13, 13] target = 5
 0 1 2 3 4 5 6 7 8 9 10 11
 L mid R
 Ans = 2

```

L = 0    R = N-1    // Define Search Space ①
while (L <= R) {
    Mid = (L+R) / 2    // Mid = L + (R-L) / 2
    if (A[Mid] == Target && (Mid == 0 || A[Mid-1] < Target))
        return Mid    // check if mid is answer ②
    if (A[Mid] < Target)
        L = Mid + 1 ✓
    else
        R = Mid - 1 ✓
}

// Decide going left or right ③
TC = O(log2(N))    SC = O(1)
return -1 ✓

```

Q → Given a sorted array of integers.

Search for a target value & return its last index, if not present return -1. (Duplicates in array)

$A = [2 \ 2 \ 5 \ 5 \ 5 \ 5 \ 8 \ 10 \ 10 \ 13 \ 13 \ 13]$ target = 10
 0 1 2 3 4 5 6 7 8 9 10 11

Ans = 8

$L = 0 \quad R = N-1$ // Define Search Space

while ($L \leq R$) { // L — R

Mid = $(L+R)/2$ // Mid = $L + (R-L)/2$

if ($A[Mid] == \text{target}$ && ($Mid == N-1$ // $A[Mid+1] > \text{target}$))

return Mid // check if mid is answer (2)

if ($A[Mid] < \text{target}$)

$L = Mid + 1$ ✓

else

$R = Mid - 1$ ✓

// Decide going left or right (3)

TC = $O(\log_2(N))$ SC = $O(1)$

}

return -1 ✓

10:25 PM

Q → Find peak element in increasing-decreasing array.

maxima
(top most)

(unique elements)

$A = [1 \ 3 \ 8 \ 10 \ 15 \ 20 \ 22 \ 19 \ 16 \ 9]$ Ans = 22
 0 1 2 3 4 5 6 7 8 9

Bruteforce → $\forall i$ check if $A[i]$ is peak element.
 $A[i-1] < A[i] > A[i+1]$

TC = $O(N)$

SC = $O(1)$

$L = 0 \quad R = N-1$

// Define search space

while ($L \leq R$) {

Mid = 0

$Mid = (L + R) / 2$
 $\rightarrow$ if $((Mid == 0 \parallel A[Mid] > A[Mid-1]) \&\& (Mid == N-1 \parallel A[Mid] > A[Mid+1]))$ // check if mid is answer.
 return $A[Mid]$

if $(Mid == 0 \parallel A[Mid] > A[Mid-1])$
 $L = Mid + 1$

$TC = O(\log_2(N))$

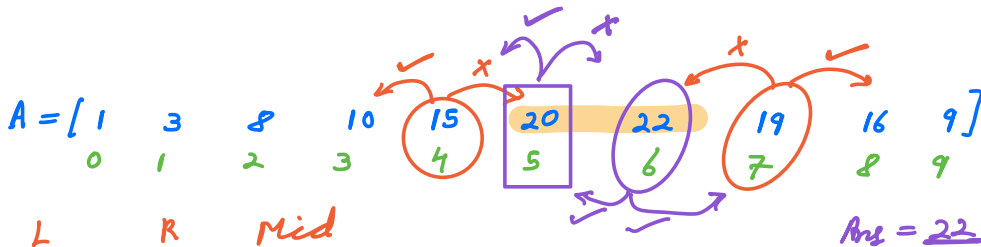
$SC = O(1)$

else

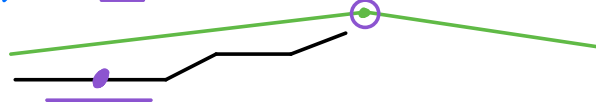
// decide to go left or right

$R = Mid - 1$

}



$A = [2, 2, 2, 2, 6, 6, 8, 7, 4, 4, 1]$



$A[m-1] \leq A[m] \geq A[m+1]$